

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples

- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Task 1 Output: Dataset Exploration Results

First five samples:

Index	variance	skewness	curtosis	entropy	class
0	3.6216	8.6661	-2.8073	-0.4470	0
1	4.5459	8.1674	-2.4586	-1.4621	0
2	3.8660	-2.6383	1.9242	0.1065	0
3	3.4566	9.5228	-4.0112	-3.5944	0
4	0.3292	-4.4552	4.5718	-0.9888	0

Dataset dimensions: 1372 rows $\times$ 5 columns (4 features + 1 target label).

Missing values: None found in any of the five columns.

Class balance: Authentic (0) = 762 samples, Forged (1) = 610 samples – a mildly imbalanced but usable binary dataset.

Descriptive statistics:

Statistic	variance	skewness	curtosis	entropy
mean	0.4337	1.9224	1.3976	-1.1917
std	2.8428	5.8690	4.3100	2.1010
min	-7.0421	-13.7731	-5.2861	-8.5482
25%	-1.7730	-1.7082	-1.5750	-2.4135
50%	0.4962	2.3197	0.6166	-0.5867
75%	2.8215	6.8146	3.1793	0.3948
max	6.8248	12.9516	17.9274	2.4495

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2–3 sentences) for every generated plot. The generated plots along with their interpretations are presented below.

7.1 Feature Histograms

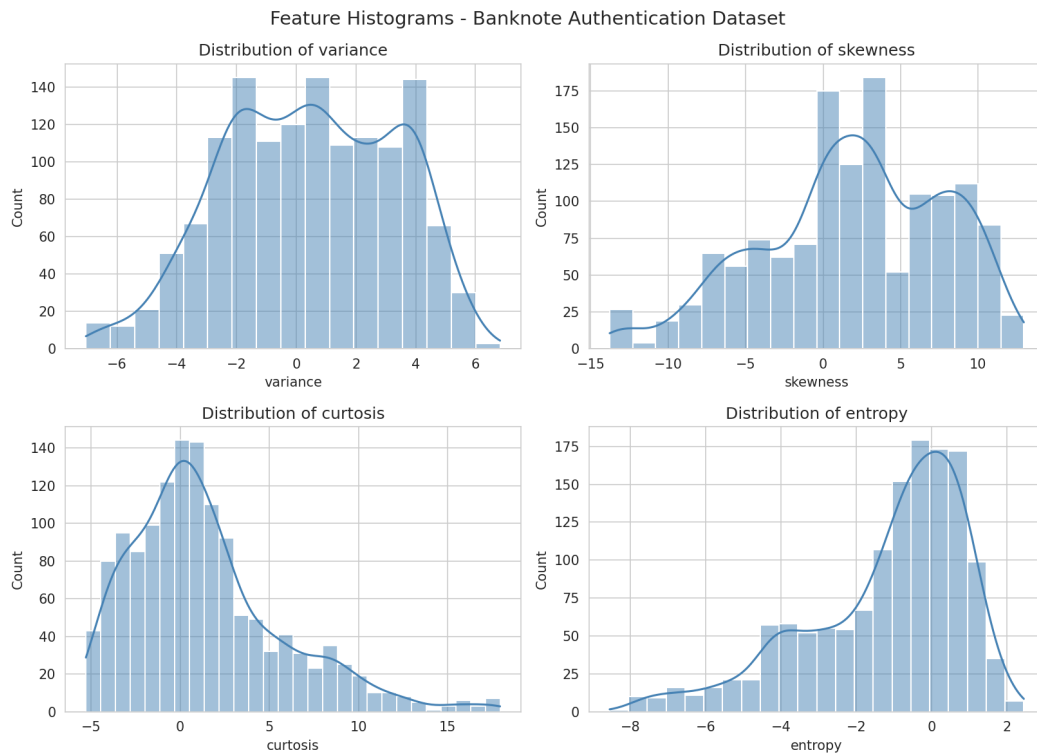


Figure 1: Distribution of the four input features.

Interpretation: **variance** shows fairly normal graph, it has around 2 to 3 peaks showing a bi or trimodal distribution which is confirmed by the skewness graph. These 2 graphs may contribute heavily towards classification as this is a binary classification. And skewness varies a lot, so it shows that a sample note had a lot of variation. While **kurtosis** is heavily towards one side, with the kurtosis graph indicating a few outlier notes with unusually high kurtosis, **entropy** is more or less around the mean.

7.2 Correlation Heatmap

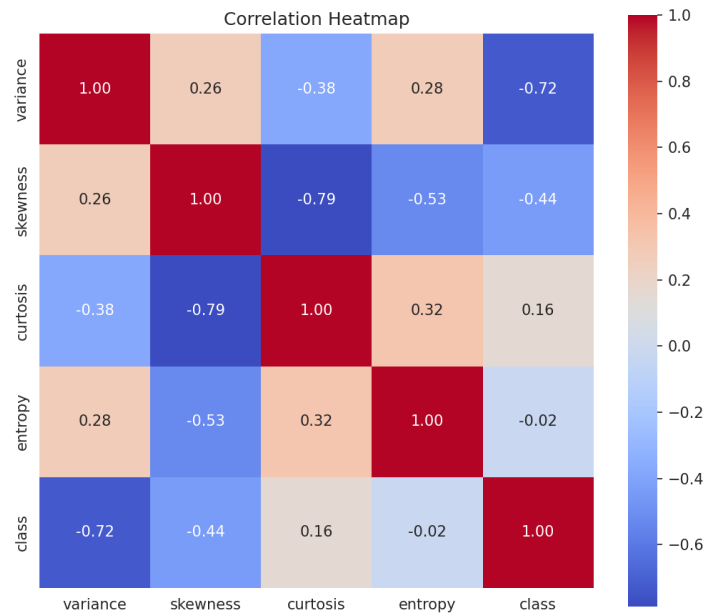


Figure 2: Correlation heatmap of the four features and the class label.

Interpretation: **variance** has the strongest negative correlation with the final class, its weight should be more influencing in the perceptron. **Skewness** and **kurtosis** are also highly negatively correlated with each other, indicating a little redundancy to identify which feature is vital for classification.

7.3 Scatter Plot

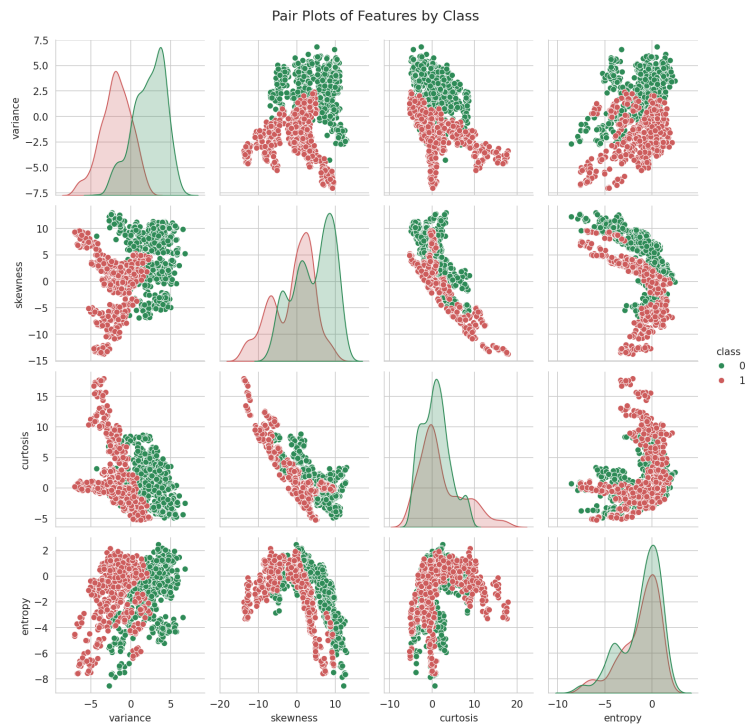


Figure 3: Class distribution using the variance and skewness features.

Interpretation: All the diagonal plots show the distributions of the feature wrt to class color and freq on the y-axis. So, the 1st feature plot seems good enough for classification as there is some overlap but generally different distributions. The 2nd feature plot and the 3rd plot seem okay but not good to classify as there is significant overlap. The 4th feature plot has maximum overlap, hence not considering it. Out of all, variance vs skewness suggests that linear classification is possible as more or less a line can separate it with few misclassified points which is good for a single Perceptron.

7.4 Boxplots

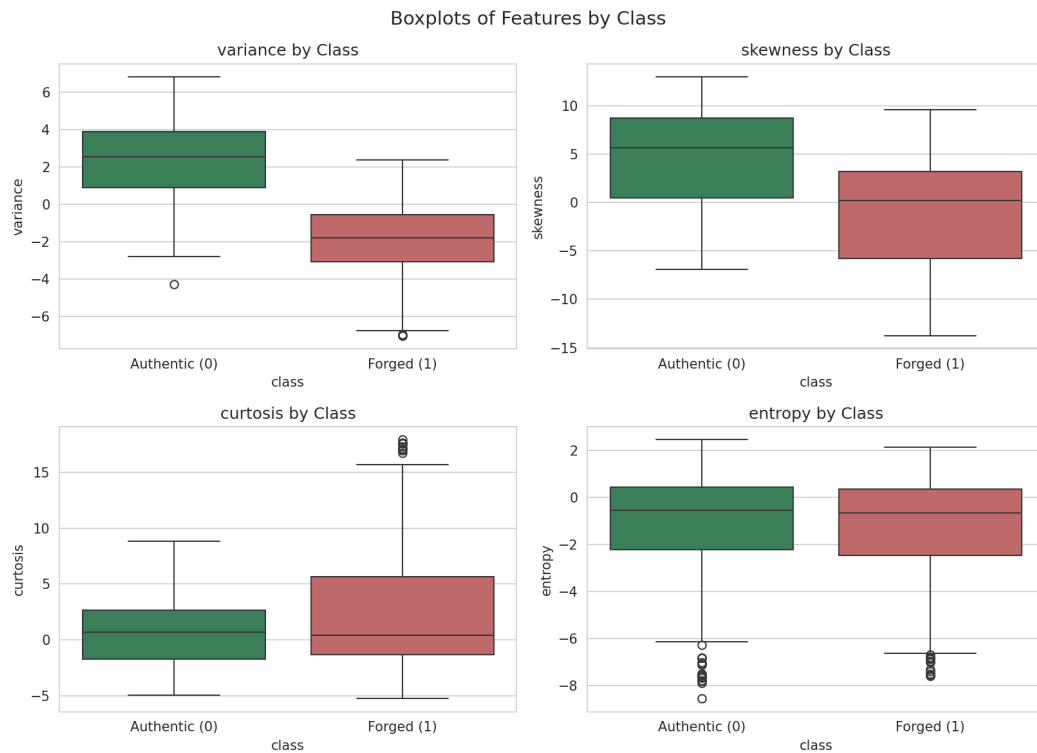


Figure 4: Boxplots of each feature grouped by class.

Interpretation: Authentic notes have higher median, whereas forged notes have lower medians and a wider spread with more outliers, more so in **kurtosis** and **entropy**. Variance doesn't have overlap like the rest of the graphs.

7.5 Training Error vs Epoch

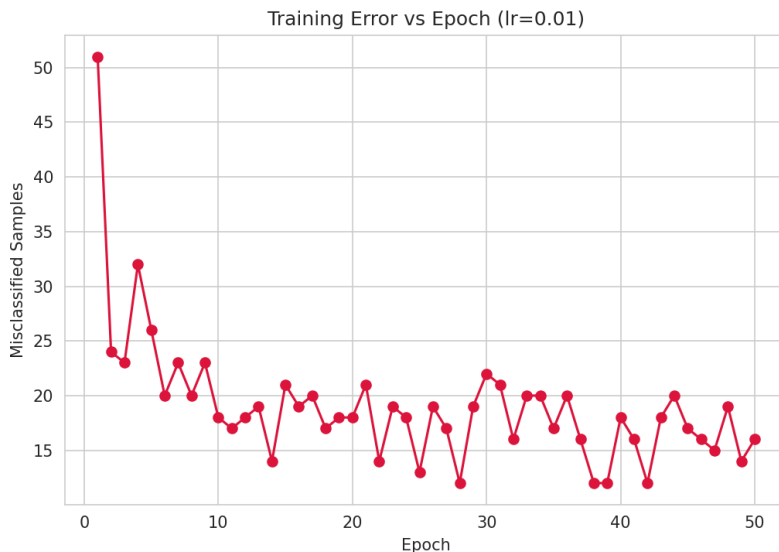


Figure 5: Number of misclassified training samples per epoch (learning rate = 0.01).

Interpretation: The error drops initially drastically but later oscillates around 15-20, indicating a minimum error convergence as there is no perfect classification here.

7.6 Weight Evolution



Figure 6: Evolution of the four weights across training epochs.

Interpretation: All weights move away from their initial values close to 0, but entropy doesn't change much showing that it doesn't have much impact. The weight for **skewness** and **kurtosis** and **variance** have the most high magnitudes as they are very informative features for the classification

7.7 Bias Evolution



Figure 7: Evolution of the bias term across training epochs.

Interpretation: Along with the weights, the bias has increased in magnitude and converged at around 40-50 epochs. So, this accommodates for the fact that there are more Authentic notes than Forged ones.

7.8 Confusion Matrix

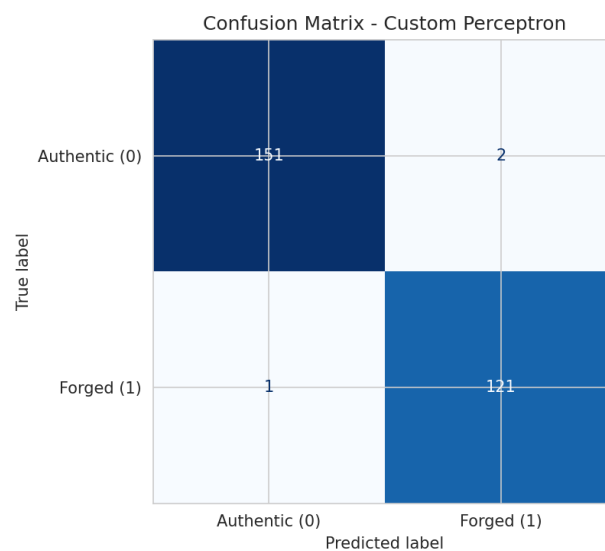


Figure 8: Confusion matrix of the custom perceptron on the test set.

Interpretation: This shows high accuracy and evaluation metrics are confirmed with this visual matrix with only single digit errors.

7.9 Learning Rate Comparison

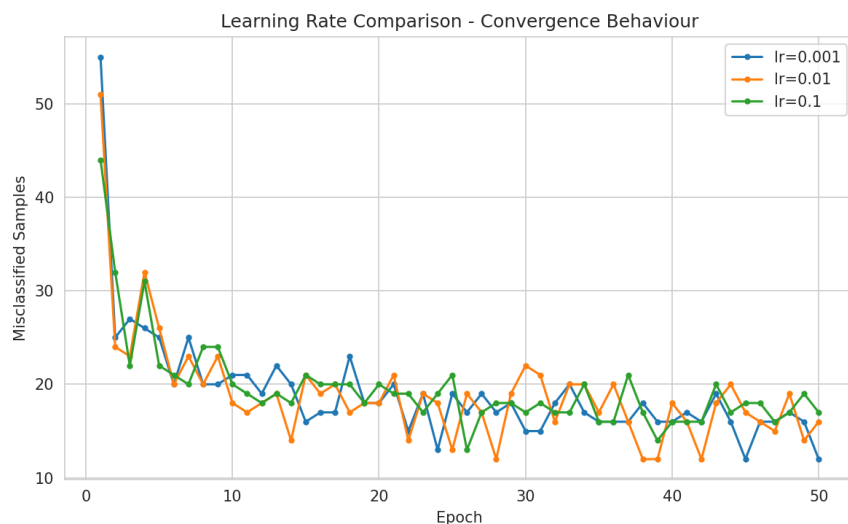


Figure 9: Training-error curves for $\eta = 0.001, 0.01, 0.1$.

Interpretation: The small lr ($\eta = 0.001$) updates weights in a slow manner, so it gives a smoother curve with minimal drastic changes. The middle lr ($\eta = 0.01$) converges to the 15-20 range the fastest among the three. The largest rate ($\eta = 0.1$) reduces the error but there are more oscillations.

7.10 Decision Boundary (Optional)

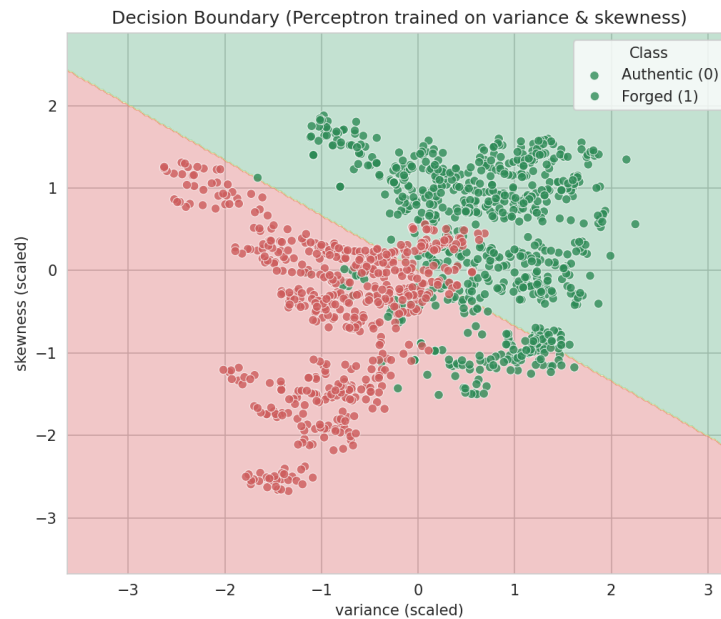


Figure 10: Decision boundary of a perceptron trained on the variance and skewness features only.

Interpretation: It shows an almost perfect visualization of the classification, and that the two features in the plot maybe enough to classify to a good enough level.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80% / 20% (1097 train / 275 test)
Learning Rate	0.01
Epochs	50
Final Weights	$[-0.1985, -0.2258, -0.2102, -0.0282]$ (variance, skewness, curtosis, entropy)
Final Bias	-0.1000
Accuracy	0.9891
Precision	0.9837
Recall	0.9918
F1-score	0.9878

Epoch-wise Learning (selected epochs)

Epoch	Errors	Weight (variance)	Weight (skewness)	Bias
1	51	-0.0626	-0.0793	-0.0300
5	26	-0.0967	-0.1214	-0.0600
10	18	-0.1199	-0.1608	-0.0600
15	21	-0.1439	-0.1819	-0.0700
20	15	-0.1595	-0.2029	-0.1000
30	19	-0.1751	-0.2126	-0.1100
40	20	-0.1876	-0.2224	-0.1000
50	16	-0.1985	-0.2258	-0.1000

Note: The full 50-epoch log (all four weights and bias per epoch) is included in the source code output and the accompanying `epoch_wise_learning.csv` file.

Learning Rate Comparison

Learning Rate	Epochs Run	Final Train Errors	Test Accuracy	Test F1-score
0.001	50	12	0.9782	0.9758
0.01	50	16	0.9891	0.9878
0.1	50	17	0.9818	0.9799

Custom Perceptron vs. Scikit-learn Perceptron

Metric	Custom Perceptron	Scikit-learn Perceptron
Accuracy	0.9891	0.9745
Precision	0.9837	0.9457
Recall	0.9918	1.0000
F1-score	0.9878	0.9721

Effect of Feature Normalization

Setup	Epochs Run	Final Train Errors	Test Accuracy
Without Normalization	50	18	0.9855
With Normalization (Standard-Scaler)	50	16	0.9891

9. Additional Tasks

9.1 Step vs. Sigmoid Activation Function

Step Activation: The Step function is discrete and discontinuous. Since the gradient is zero and non differentiable at zero, it can't be used with gradient based optimizers and there cannot be any learning or updating of weights. Hence, it is mostly used in single Perceptron only. On the other hand, The sigmoid function brings a continuous range of values from 0 to 1. It is differentiable and brings a range of values into the output and can handle complex relationships.

9.2 Comparison with Scikit-learn's Perceptron

As shown in the table, the custom Perceptron shows marginally higher accuracy than the Scikit one. And the Precision and F1 score is also a little higher than the latter but Recall is 100 percent in Scikit Learn. So, there are similar results but few variations due to diff initialized weights and stopping point, and epoch wise shuffling, etc.

9.3 Learning Rates 0.001, 0.01, 0.1

There is not much difference between the learning rate scores, but the moderate one performed slightly better than the other two indication of the fact that the other two were too high or too low.

9.4 Why a Single Layer Perceptron Cannot Solve XOR

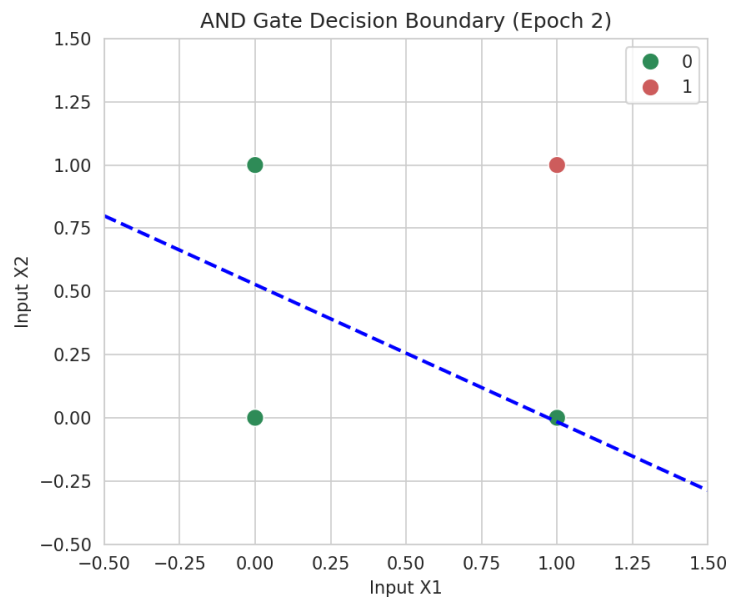
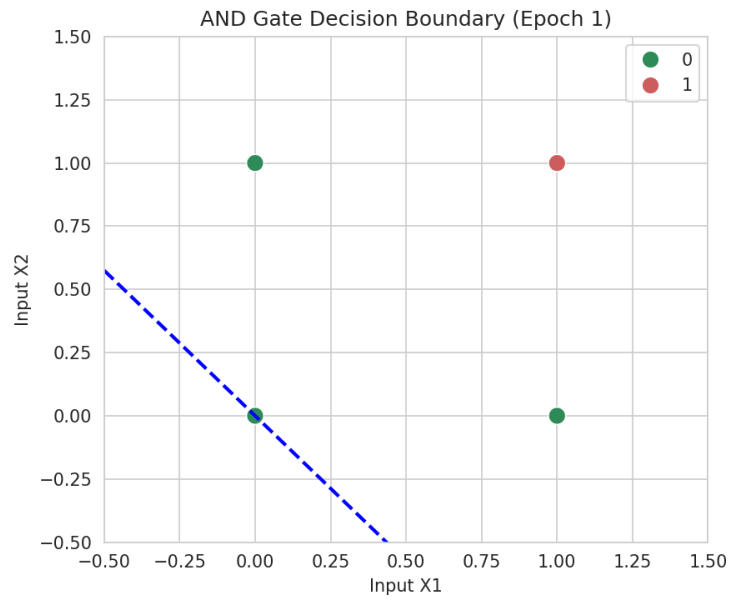
The XOR function is defined as: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$.

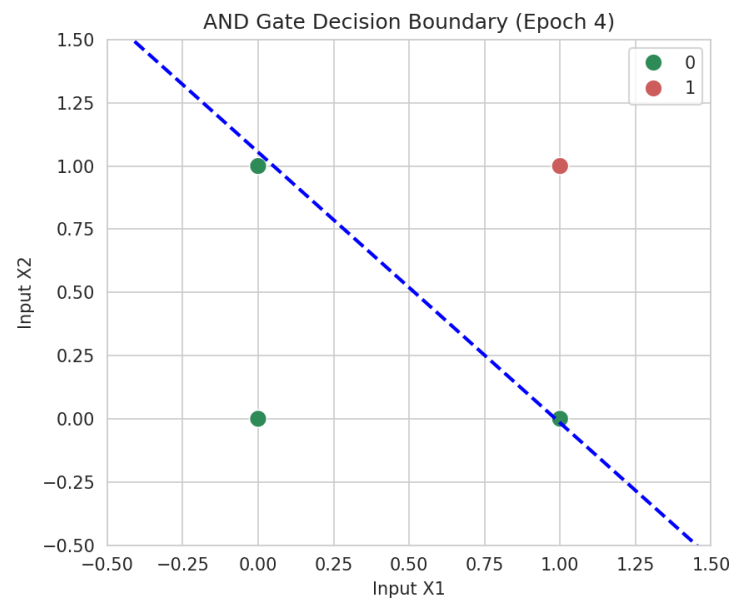
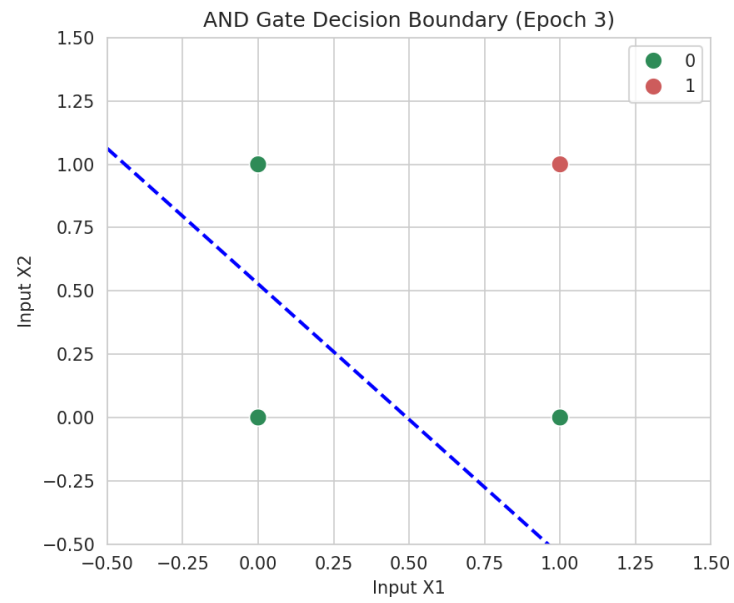
A Single Perceptron can only classify linearly, whereas The XOR requires non linear boundary as it has points plotted in a diagonally opposite way. Hence, Single Perceptron cannot solve XOR Problem. A small demonstration is also done in the code with 20 epochs to train a model with a learning rate of 0.1 but the Perceptron misclassifies all the points and never converges.

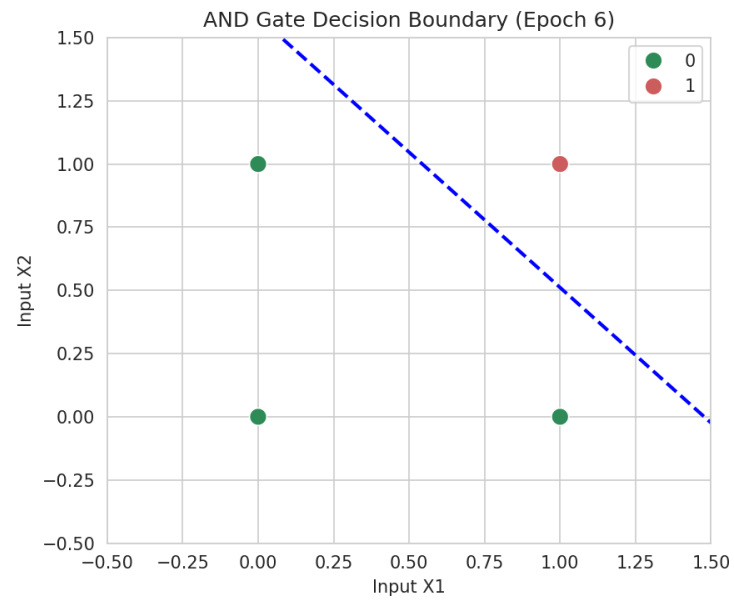
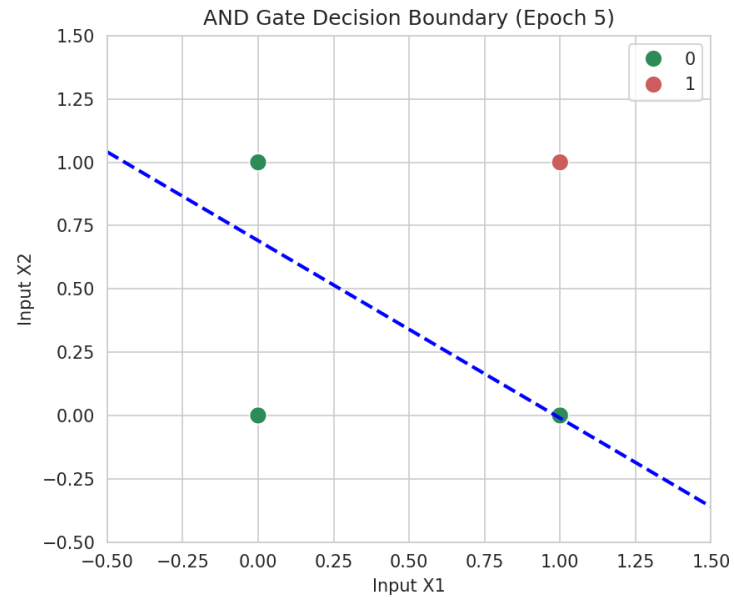
9.5 Effect of Feature Normalization on Convergence

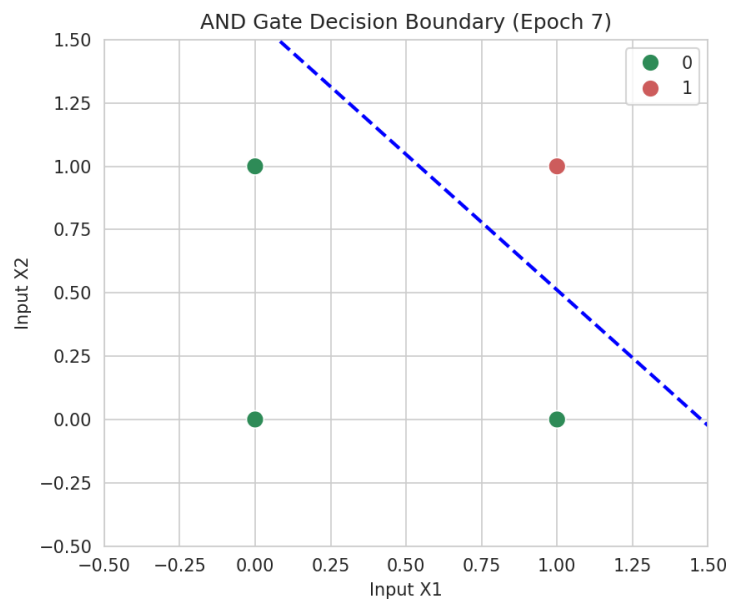
See the Normalization comparison table in Section 8. Without normalization, the features have very different scales – e.g. `curtosis` ranges up to 17.9 while `entropy` ranges from -8.5 to 2.4) caused the weighted sum, and hence the learning updates, to be dominated by high magnitude features, resulting in more training errors (18) and lower test accuracy (0.9855). After normalizing with `StandardScaler`, all features contributed more evenly to the weighted sum, giving fewer final training errors (16) and higher test accuracy (0.9891) within the same 50 epochs. Since, it is a small experiment, the difference was not substantial.

9.6 Code the perceptron learning algorithm for OR, NOT, and AND gates using Python



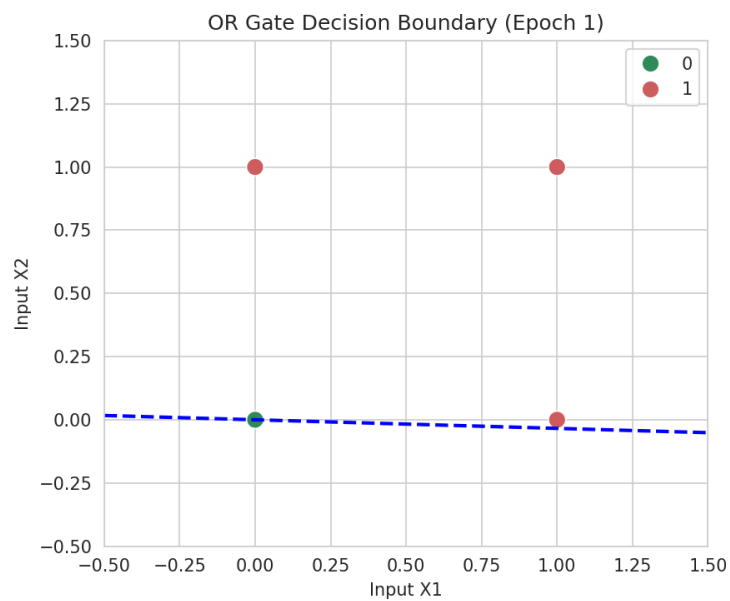


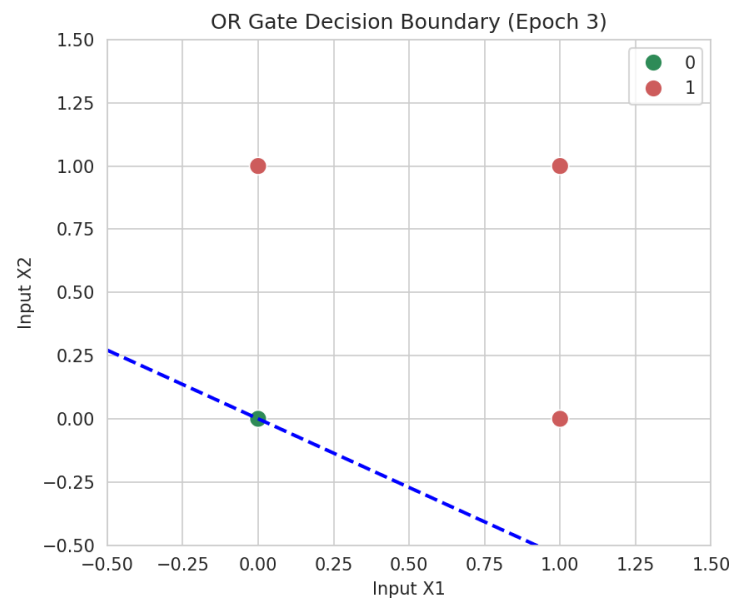
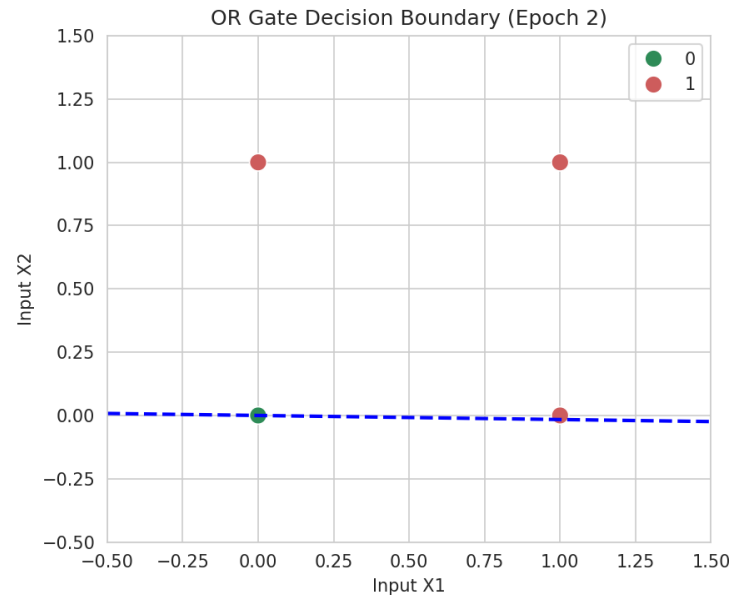


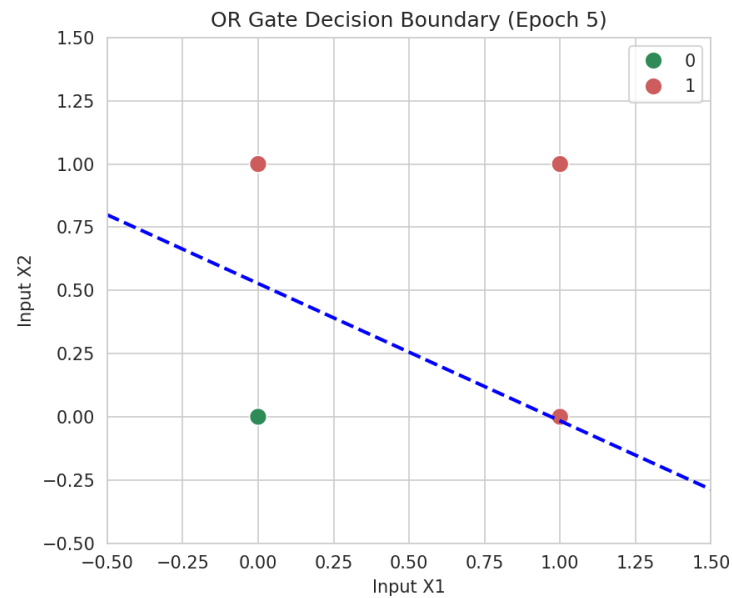
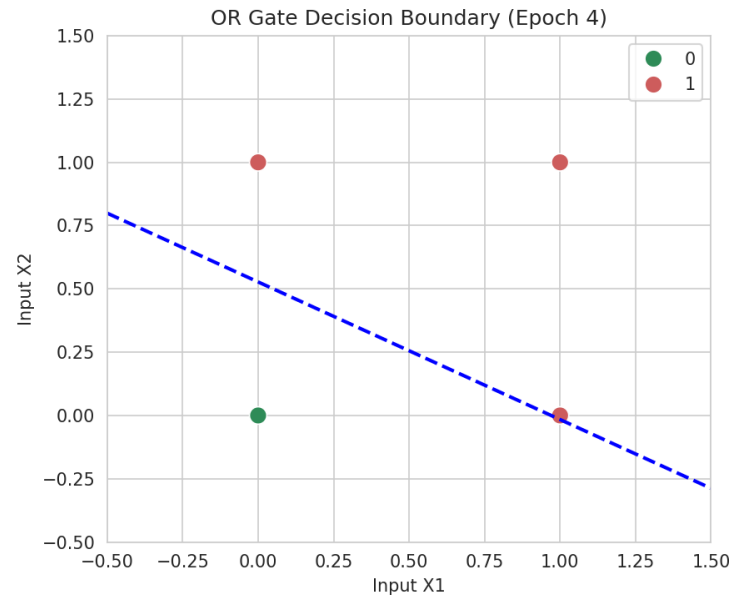


AND Gate

The above photos show the epochs while training a Simple Perceptron to classify the AND points which has true for only both inputs as 1 otherwise false(Binary classes). As we can see from the plot, the points are easily linearly classifiable. Hence, not much epochs are needed and 100% accuracy is achieved here.

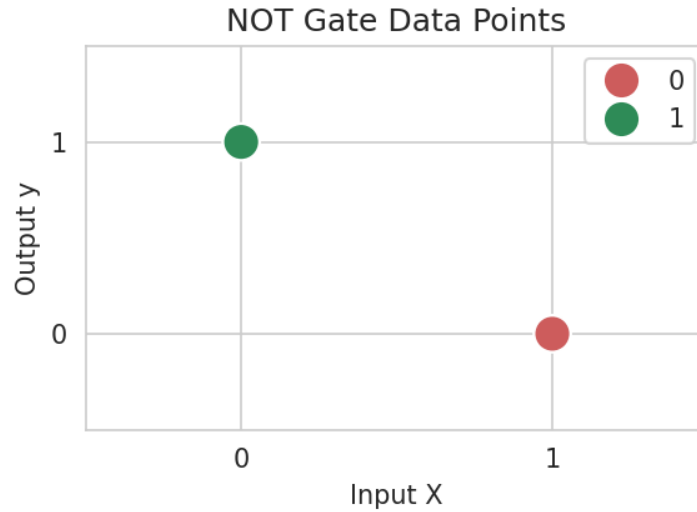






OR Gate

The above photos show the epochs while training a Simple Perceptron to classify the OR points which has true for atleast any of the inputs as 1 otherwise false(Binary classes). As we can see from the plot, the points are easily linearly classifiable. Hence, not much epochs are needed and 100% accuracy is achieved here.



NOT Gate

The above photo shows the NOT gate points plotted in x and y space. Here, as there are only two points, there is no need of a line required to classify them. A point is enough to classify them. Hence, no meaningful decision boundary is shown.

10. Source Code

The complete source code is available on:

<https://colab.research.google.com/drive/1H6vEFQ01eKGqYJ060kR-3paRTkt1nH59?usp=sharing>

11. Discussion

The Single Layer Perceptron achieved great performance with very high accuracy as found in the EDA Scatter plots which showed that the data was almost linearly classifiable.

The epoch wise training graph shows that there is a significant initial drop but it starts oscillating in the 15-20 range and does not converge to perfect errorless score as the overlap is seen in the box-plots and the scatter plots plotted earlier.

The learning-rate η mainly influences the speed and size of the learning rather than the learning part itself. $\eta = 0.01$ gave the best balance, while $\eta = 0.001$ slightly under corrected and $\eta = 0.1$ slightly over-corrected.

Scikit-learn's **Perceptron** shows similar results as to the Custom **Perceptron**, both reaching comparable accuracy and F1-score using the same underlying update rule, with small differences due to implementation details such as sample shuffling, stopping criteria, etc.

Finally, the normalization shows the fundamental property of the perceptron: it is sensitive to feature scale due to the weighted sum is a simple linear combination.

12. Conclusion

So to conclude, in this experiment a Custom Perceptron was used to classify a non-perfect linearly classifiable dataset. It showed very good results with high accuracy and metrics which were comparable to Scikit Learn's Perceptron. Around 50 epochs were chosen which was appropriate given the size and nature of the dataset. And the error rate oscillated to a range and did not further dip given the dataset was not perfectly linearly classifiable. The Learning rate when kept moderate showed better results, and similarly Normalization proved to be a vital part as it scaled the features accordingly which resulted in better weight updation and finally, better accuracy. And implementation of training a Simple Perceptron for AND, OR and NOT gates was very easy with lesser epochs and 100% accuracy achieved during the experiment.

14. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.